DSA (Data Structures & Algorithms) Roadmap

Topic Order & Timeline

```
Week 1:     Complexity Analysis
Week 2-3:   Arrays & Strings
Week 3-4:   Linked Lists
Week 4:     Stacks & Queues, Recursion
Week 4-5:   Sorting Algorithms
Week 5-6:   Trees (Binary Tree, BST)
Week 6:     Heaps / Priority Queues
Week 6-7:   Hashing
Week 7-8:   Graphs (BFS, DFS)
Week 8:     Binary Search, Two Pointers, Sliding Window
Week 9-10:  Dynamic Programming
Week 10:    Backtracking
Week 11:    Greedy, Tries, Advanced Trees
Week 12-13: Mixed Practice & Mock Interviews
```

---

1. Complexity Analysis

- [ ] Big-O, Big-Theta, Big-Omega notation
- [ ] Time complexity analysis (loops, recursion)
- [ ] Space complexity analysis
- [ ] Best, worst, average case
- [ ] Amortized analysis (concept)

Practice: Analyze complexity of given code snippets

---

2. Arrays & Strings

- [ ] Array operations: insert, delete, search, traverse
- [ ] Two pointer technique
- [ ] Sliding window technique
- [ ] Prefix sum
- [ ] Kadane's algorithm (max subarray)
- [ ] String manipulation & pattern matching

Must-Solve Problems:

- [ ] Two Sum
- [ ] Best Time to Buy and Sell Stock
- [ ] Contains Duplicate
- [ ] Maximum Subarray
- [ ] Product of Array Except Self

- [ ] 3Sum
- [ ] Container With Most Water
- [ ] Valid Anagram
- [ ] Longest Substring Without Repeating Characters
- [ ] Group Anagrams

---

3. Linked Lists

- [ ] Singly linked list (insert, delete, search, reverse)
- [ ] Doubly linked list
- [ ] Circular linked list
- [ ] Fast & slow pointer technique

Must-Solve Problems:

- [ ] Reverse Linked List
- [ ] Merge Two Sorted Lists
- [ ] Linked List Cycle
- [ ] Remove Nth Node From End
- [ ] Reorder List
- [ ] Add Two Numbers
- [ ] Find Middle of Linked List

---

4. Stacks & Queues

- [ ] Stack: array & linked list implementation
- [ ] Queue: array & linked list implementation
- [ ] Circular queue
- [ ] Monotonic stack
- [ ] Queue using two stacks

Must-Solve Problems:

- [ ] Valid Parentheses
- [ ] Min Stack
- [ ] Next Greater Element
- [ ] Daily Temperatures
- [ ] Implement Queue using Stacks
- [ ] Largest Rectangle in Histogram

---

5. Recursion & Sorting

- [ ] Recursion basics, base case, recursive case
- [ ] Tail recursion
- [ ] Bubble Sort, Selection Sort, Insertion Sort
- [ ] Merge Sort (divide & conquer)
- [ ] Quick Sort (partition scheme)

- [ ] Counting Sort, Radix Sort (concept)

Must-Solve Problems:

- [ ] Fibonacci (recursive + memoized)
- [ ] Power of a number
- [ ] Sort an Array (implement merge sort)
- [ ] Sort Colors (Dutch National Flag)
- [ ] Merge Intervals

---

6. Trees

- [ ] Binary Tree: representation, traversals (inorder, preorder, postorder)
- [ ] Level order traversal (BFS)
- [ ] BST: insert, search, delete
- [ ] Height, diameter, balanced check
- [ ] Lowest Common Ancestor
- [ ] Serialize/Deserialize tree

Must-Solve Problems:

- [ ] Maximum Depth of Binary Tree
- [ ] Invert Binary Tree
- [ ] Same Tree
- [ ] Subtree of Another Tree
- [ ] Validate BST
- [ ] Kth Smallest Element in BST
- [ ] Binary Tree Level Order Traversal
- [ ] Lowest Common Ancestor of BST
- [ ] Binary Tree Right Side View
- [ ] Diameter of Binary Tree

---

7. Heaps / Priority Queues

- [ ] Min heap, max heap
- [ ] Heap operations: insert, extract, heapify
- [ ] Priority queue usage
- [ ] Heap sort

Must-Solve Problems:

- [ ] Kth Largest Element in Array
- [ ] Top K Frequent Elements
- [ ] Merge K Sorted Lists
- [ ] Find Median from Data Stream
- [ ] Task Scheduler

---

8. Hashing

- ☐ Hash functions, collision handling
- ☐ Hash map / Hash set operations
- ☐ Frequency counting
- ☐ Anagram detection

Must-Solve Problems:

- ☐ Two Sum (hash approach)
- ☐ Group Anagrams
- ☐ Longest Consecutive Sequence
- ☐ Subarray Sum Equals K
- ☐ Top K Frequent Elements
- ☐ Valid Sudoku

---

9. Graphs

- ☐ Representation: adjacency list, adjacency matrix
- ☐ BFS (breadth-first search)
- ☐ DFS (depth-first search)
- ☐ Cycle detection (directed & undirected)
- ☐ Topological sort
- ☐ Dijkstra's algorithm
- ☐ Bellman-Ford algorithm
- ☐ Union-Find (Disjoint Set Union)
- ☐ Minimum Spanning Tree (Prim's / Kruskal's)

Must-Solve Problems:

- ☐ Number of Islands
- ☐ Clone Graph
- ☐ Pacific Atlantic Water Flow
- ☐ Course Schedule (I & II)
- ☐ Graph Valid Tree
- ☐ Number of Connected Components
- ☐ Word Ladder
- ☐ Network Delay Time (Dijkstra)

---

10. Binary Search

- ☐ Basic binary search (iterative & recursive)
- ☐ Binary search on answer
- ☐ Lower bound, upper bound
- ☐ Search in rotated sorted array

Must-Solve Problems:

- [ ] Binary Search
- [ ] Search in Rotated Sorted Array
- [ ] Find Minimum in Rotated Sorted Array
- [ ] Search a 2D Matrix
- [ ] Koko Eating Bananas
- [ ] Median of Two Sorted Arrays

---

11. Dynamic Programming

- [ ] Memoization (top-down)
- [ ] Tabulation (bottom-up)
- [ ] 1D DP patterns
- [ ] 2D DP patterns
- [ ] Knapsack variants
- [ ] String DP (LCS, edit distance)
- [ ] DP on trees (concept)

Must-Solve Problems:

- [ ] Climbing Stairs
- [ ] House Robber (I & II)
- [ ] Coin Change
- [ ] Longest Increasing Subsequence
- [ ] Longest Common Subsequence
- [ ] Word Break
- [ ] 0/1 Knapsack
- [ ] Edit Distance
- [ ] Unique Paths
- [ ] Decode Ways
- [ ] Partition Equal Subset Sum
- [ ] Target Sum

---

12. Backtracking

- [ ] Backtracking template
- [ ] Pruning techniques
- [ ] State space tree

Must-Solve Problems:

- [ ] Subsets (I & II)
- [ ] Permutations (I & II)
- [ ] Combination Sum (I & II)
- [ ] N-Queens
- [ ] Sudoku Solver
- [ ] Word Search
- [ ] Palindrome Partitioning

---

13. Greedy Algorithms

- ☐ Greedy choice property
- ☐ Activity selection
- ☐ Interval scheduling

Must-Solve Problems:

- ☐ Jump Game (I & II)
- ☐ Gas Station
- ☐ Meeting Rooms (I & II)
- ☐ Merge Intervals
- ☐ Non-overlapping Intervals

---

14. Tries

- ☐ Trie implementation (insert, search, startsWith)
- ☐ Applications: autocomplete, spell check

Must-Solve Problems:

- ☐ Implement Trie
- ☐ Word Search II
- ☐ Design Add and Search Words

Total Problem Count Target

Difficulty	Count
Easy	60+
Medium	70+
Hard	15+
Total	**150+**

LeetCode Problem Tracking

#	Problem	Difficulty	Topic	C++	Python	Date
1	Two Sum	Easy	Array/Hash	☐	☐	
2	Add Two Numbers	Medium	Linked List	☐	☐	

Add more as you solve